

Machine Learning

Unidad 3 — Fundamentos matemáticos, algoritmos clásicos, evaluación y buenas prácticas.

Contenido

01	Álgebra lineal para ML
02	Descenso de gradiente
03	Teorema de Bayes
04	Análisis exploratorio de datos (EDA)
05	Preprocesamiento de datos
06	Introducción a Machine Learning

07	Aprendizaje supervisado: regresión avanzada
08	Aprendizaje supervisado: clasificación avanzada
09	Aprendizaje no supervisado
10	Modelos de ensamble
11	Pipelines y evaluación de modelos

01

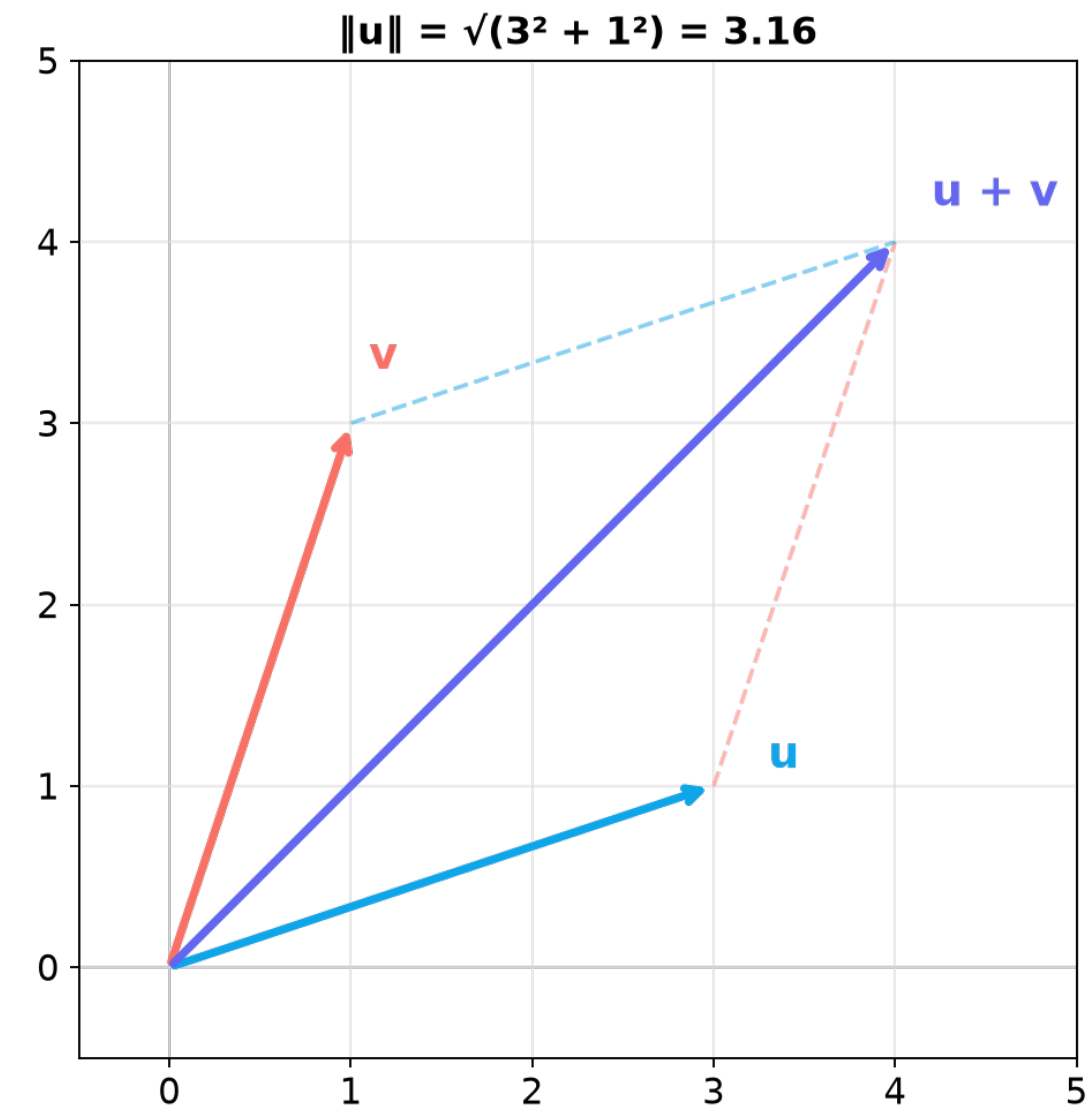
**Álgebra Lineal
para ML**

Vectores y su norma

Un vector representa, en ML, una fila de datos: cada característica es una componente

Norma (magnitud): $\|v\| = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$

- Espacio vectorial: el conjunto de vectores que se pueden formar combinando una base
- Base: el conjunto mínimo de vectores que genera todo el espacio
- Dimensión: cuántos vectores tiene la base — en ML, el número de características



Operaciones con matrices

Operación	Regla	Uso típico en ML
Suma / resta	Elemento a elemento, mismas dimensiones	Combinar o comparar conjuntos de pesos
Producto punto	$u \cdot v = \sum u_i v_i$	Similitud entre dos vectores de características
Producto matricial	$(AB)_{ij} = \sum_k A_{ik} B_{kj}$	Propagar datos a través de una capa de una red
Producto matriz-vector	Ax = combinación lineal de las columnas de A	Aplicar un modelo lineal a una fila de datos
Transpuesta / inversa	A^T intercambia filas y columnas; A^{-1} deshace A	Resolver sistemas de ecuaciones normales

Toda la matemática de una red neuronal (Unidad 4) se reduce, en el fondo, a productos matriciales encadenados.

02

**Descenso de
Gradiente**

Cómo funciona el descenso de gradiente

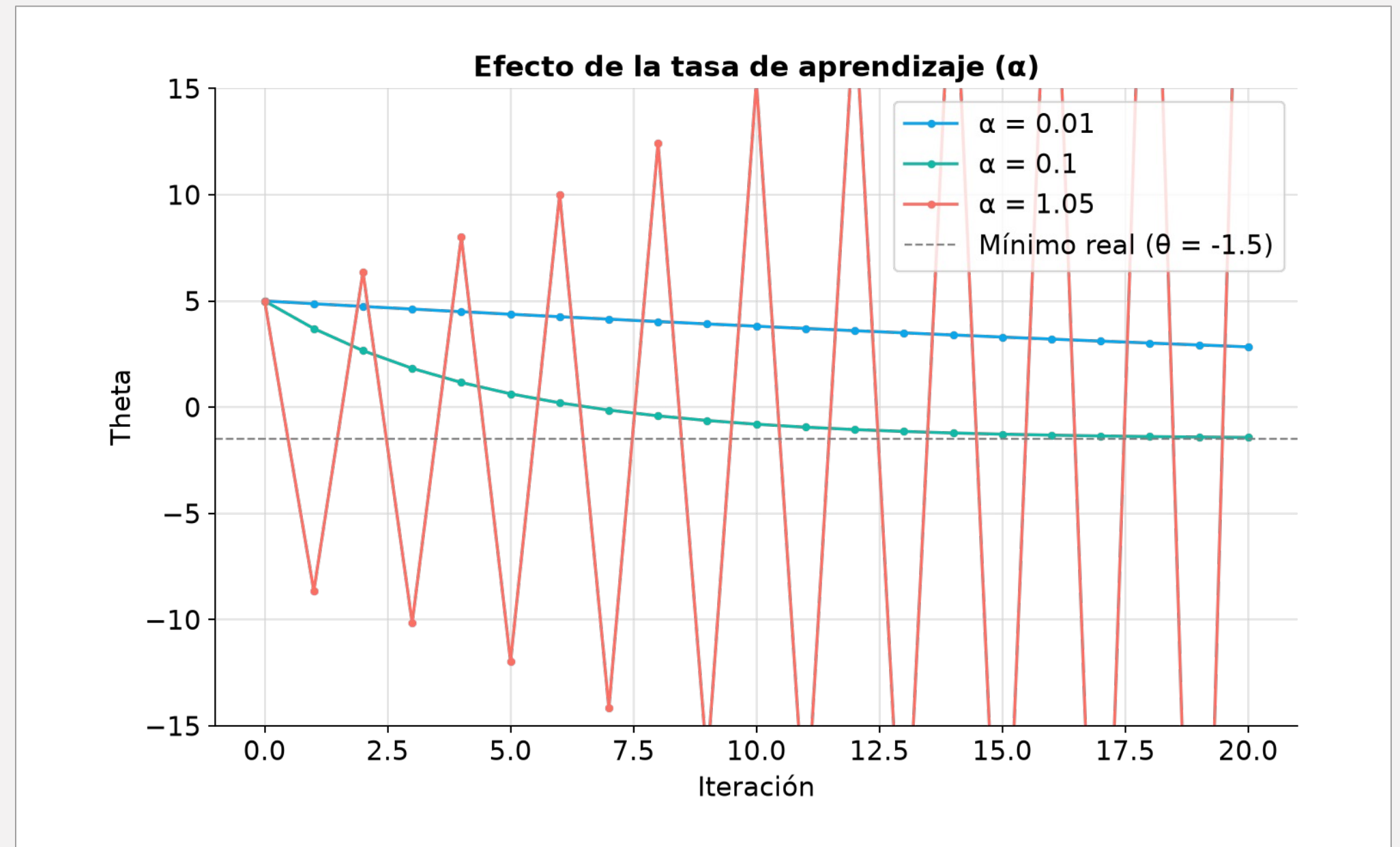
- Función de costo $J(\theta)$: mide qué tan lejos están las predicciones de los valores reales
- Entrenar = encontrar los parámetros θ que minimizan $J(\theta)$
- α (tasa de aprendizaje): qué tan grande es cada paso hacia el mínimo
- Ejemplo real del notebook: $J(\theta) = \theta^2 + 3\theta + 2$, mínimo en $\theta = -1.5$

$$\theta_{\text{nuevo}} = \theta_{\text{actual}} - \alpha \cdot J'(\theta_{\text{actual}})$$

Regla de actualización del descenso de gradiente.

El efecto real de la tasa de aprendizaje

- $\alpha = 0.01$: converge, pero muy lento — $\theta=2.84$ tras 20 iteraciones, lejos aún de -1.5
- $\alpha = 0.1$: converge rápido y estable — $\theta=-1.43$, muy cerca del mínimo real
- $\alpha = 1.05$: diverge — oscila y crece sin control ($\theta=42.2$, costo > 1900)

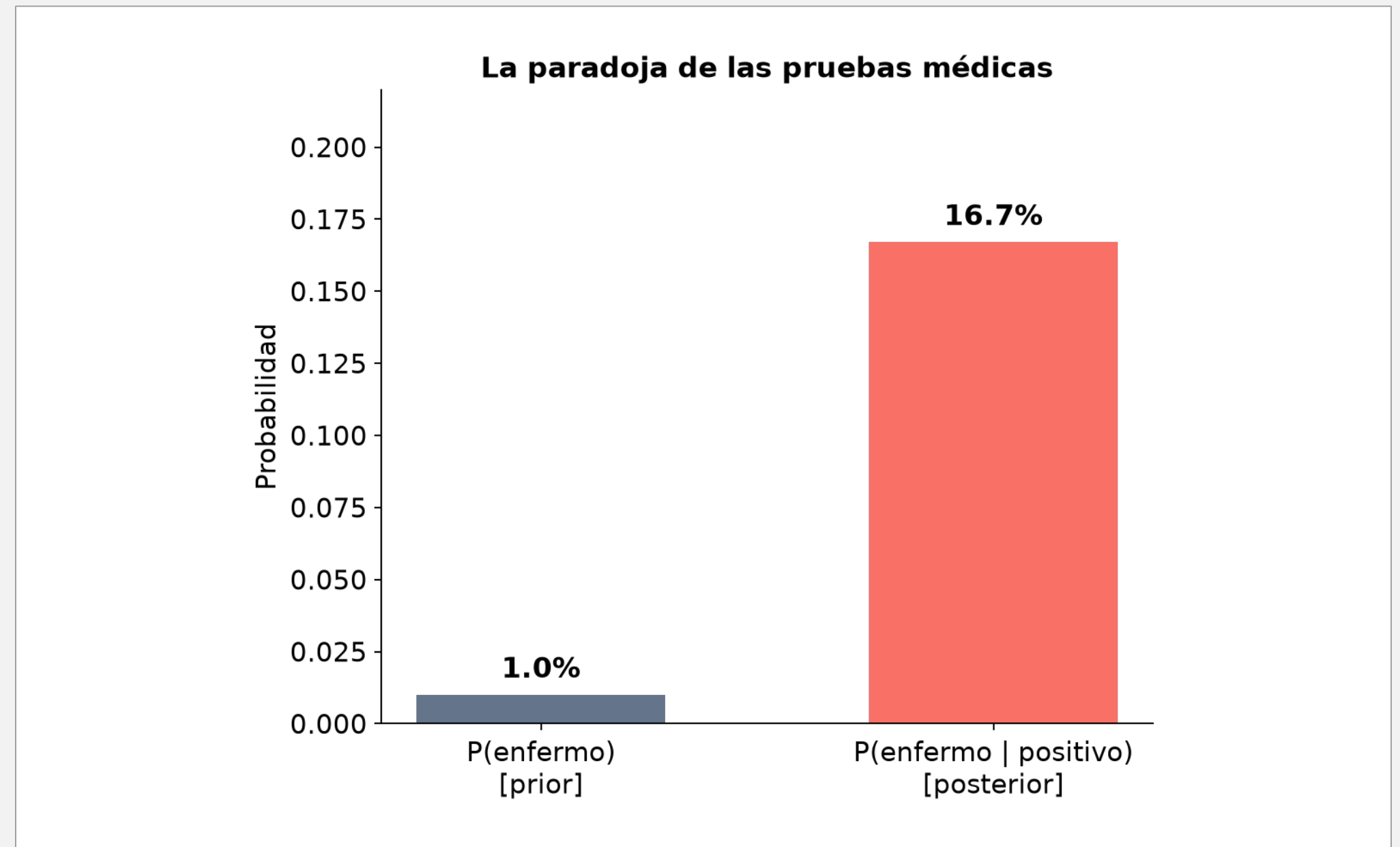


03

Teorema de Bayes

La paradoja de las pruebas médicas

- Prevalencia: 1% de la población tiene la enfermedad
- Sensibilidad: 99% de positivos entre los enfermos
Especificidad: 95% de negativos entre los sanos
- La mayoría intuye "90-95%" — el resultado real es 16.7%
- Como la enfermedad es rara, la mayoría de los positivos son falsos positivos del enorme grupo de personas sanas

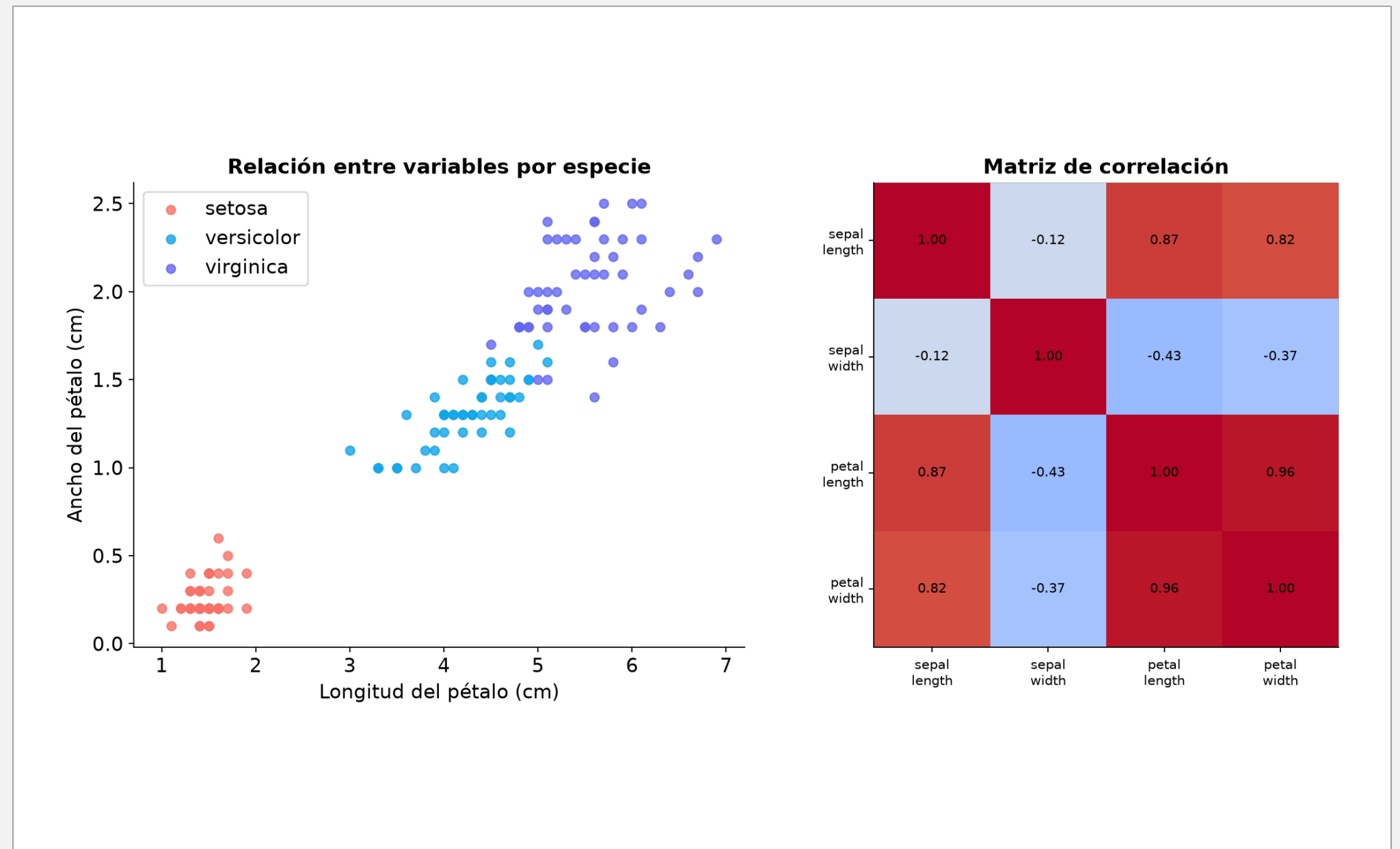


04

Análisis Exploratorio de Datos (EDA)

El flujo de trabajo de un EDA, sobre Iris

- Flujo: cargar → verificar (nulos, tipos) → describir → visualizar → correlacionar
- Las 3 especies se separan casi perfectamente por longitud y ancho del pétalo
- Se confirma en la matriz de correlación (0.96 entre ambas variables)



05

Preprocesamiento de Datos

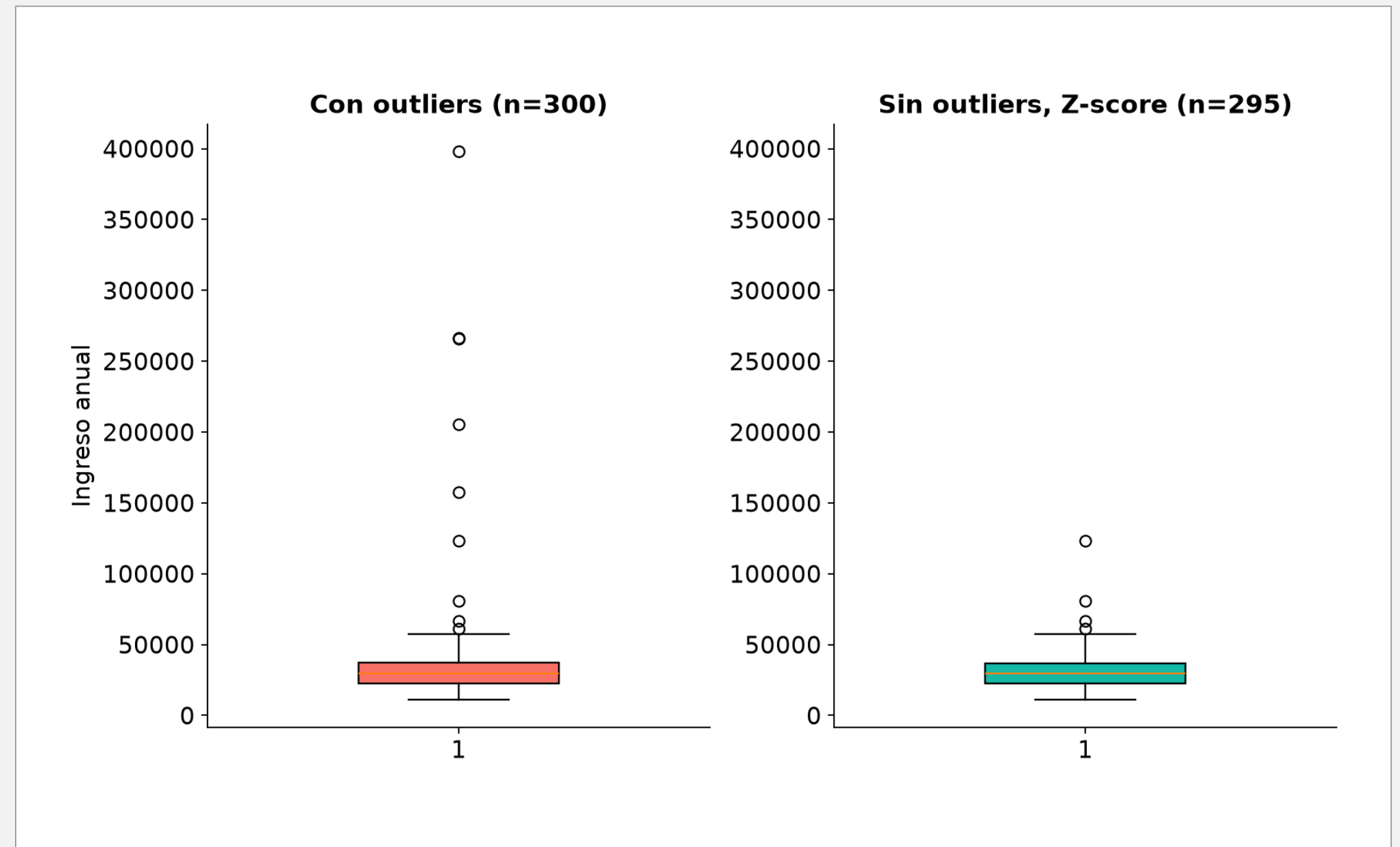
Limpieza, outliers y transformación

Problema	Técnicas vistas
Valores faltantes	Eliminación (dropna) o imputación (media/mediana/moda)
Outliers	Detección: Z-score, boxplot · Tratamiento: eliminación con rango intercuartílico (IQR)
Distribuciones sesgadas	Escalado logarítmico, winsorización (recortar valores extremos)

Dataset de trabajo: solicitudes de crédito bancario, generado sintéticamente con outliers genuinos inyectados a propósito.

Outliers reales: antes y después del Z-score

- 6 solicitudes con ingresos anormalmente altos
(7× el resto) se inyectaron a propósito
- El filtro $|Z| > 3$ los elimina correctamente



Escalado, codificación, PCA y validación

Técnica	Qué resuelve
Normalización (Min-Max)	Lleva cada variable al rango [0,1]
Estandarización (Z-score)	Media 0, desviación estándar 1 — necesario cuando las variables tienen escalas muy distintas
Codificación ordinal	Variables categóricas con un orden natural (p. ej. nivel educativo)
One-Hot Encoding	Variables categóricas sin orden (p. ej. ciudad)
PCA	Reduce dimensionalidad conservando la mayor varianza posible
Cross-validation / train-val-test	Evita la fuga de datos (data leakage) entre entrenamiento y evaluación

Regla de oro: todo transformador (escalador, codificador, PCA) se ajusta (fit) solo con los datos de entrenamiento.

06

**Introducción a
Machine Learning**

Aprendizaje supervisado: regresión y clasificación

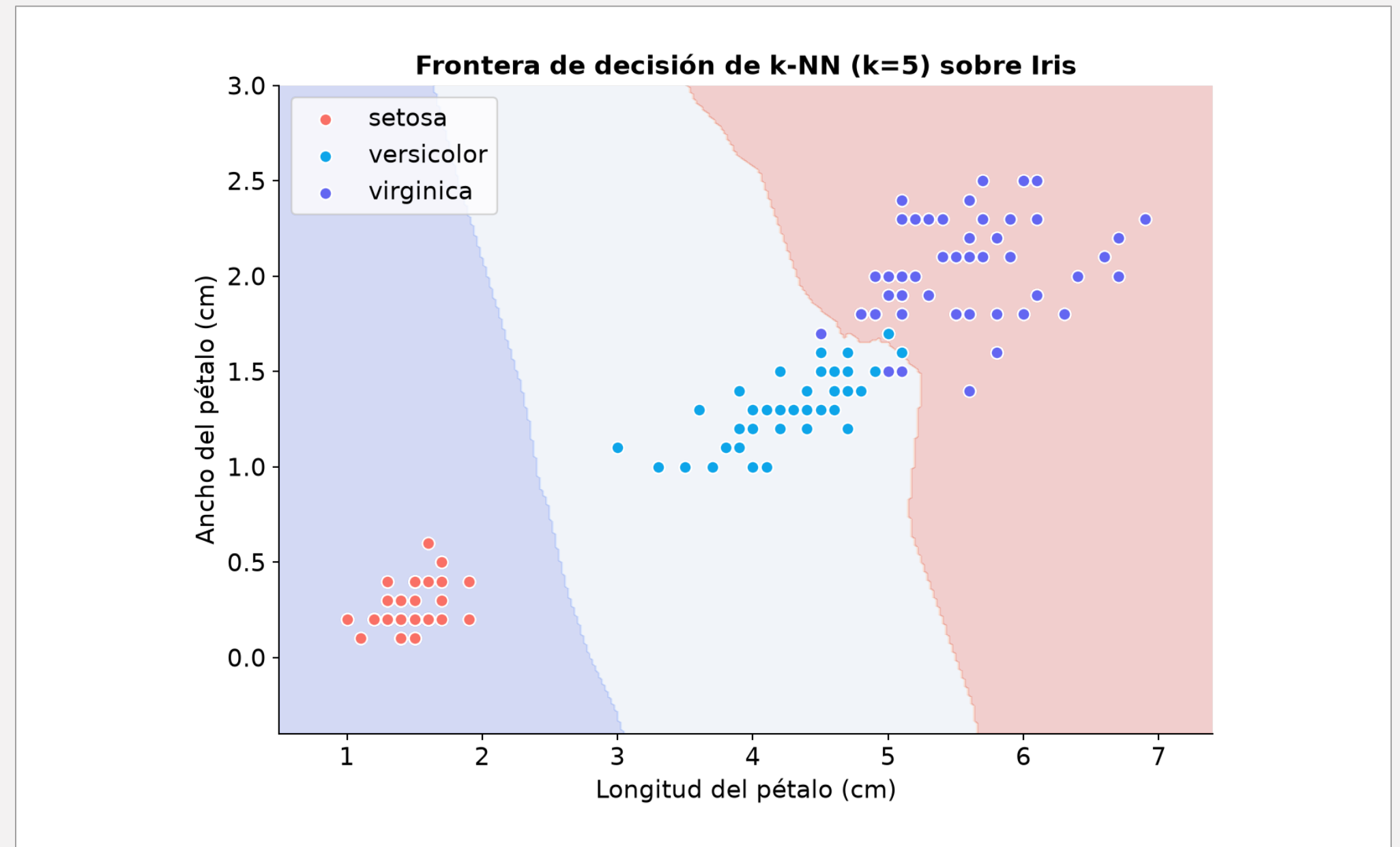
Tarea	Modelos vistos	Dataset de ejemplo
Regresión	Regresión lineal	California Housing (predecir precio de vivienda)
Clasificación	k-NN, Árboles de Decisión	Iris (4 variables, 3 clases bien separadas)

k-NN clasifica un punto nuevo según la clase mayoritaria entre sus k vecinos más cercanos.

La frontera de decisión de k-NN, en la práctica

k=5: cada región del plano se colorea según la clase mayoritaria entre los 5 vecinos más cercanos

Con Iris, incluso una regla tan simple separa casi perfectamente las 3 especies



No supervisado, ensambles y AutoML

Bloque	Qué se ve
No supervisado	Clustering con K-Means — agrupar datos sin etiquetas previas
Ensambles	Random Forest (bagging de árboles), XGBoost (boosting)
AutoML	PyCaret: compara automáticamente docenas de modelos con pocas líneas de código
Optimización de hiperparámetros	Grid Search (prueba exhaustiva) y Randomized Search (muestreo aleatorio del espacio de búsqueda)

Este notebook es el recorrido panorámico del curso: cada bloque se retoma y profundiza en los notebooks siguientes.

07

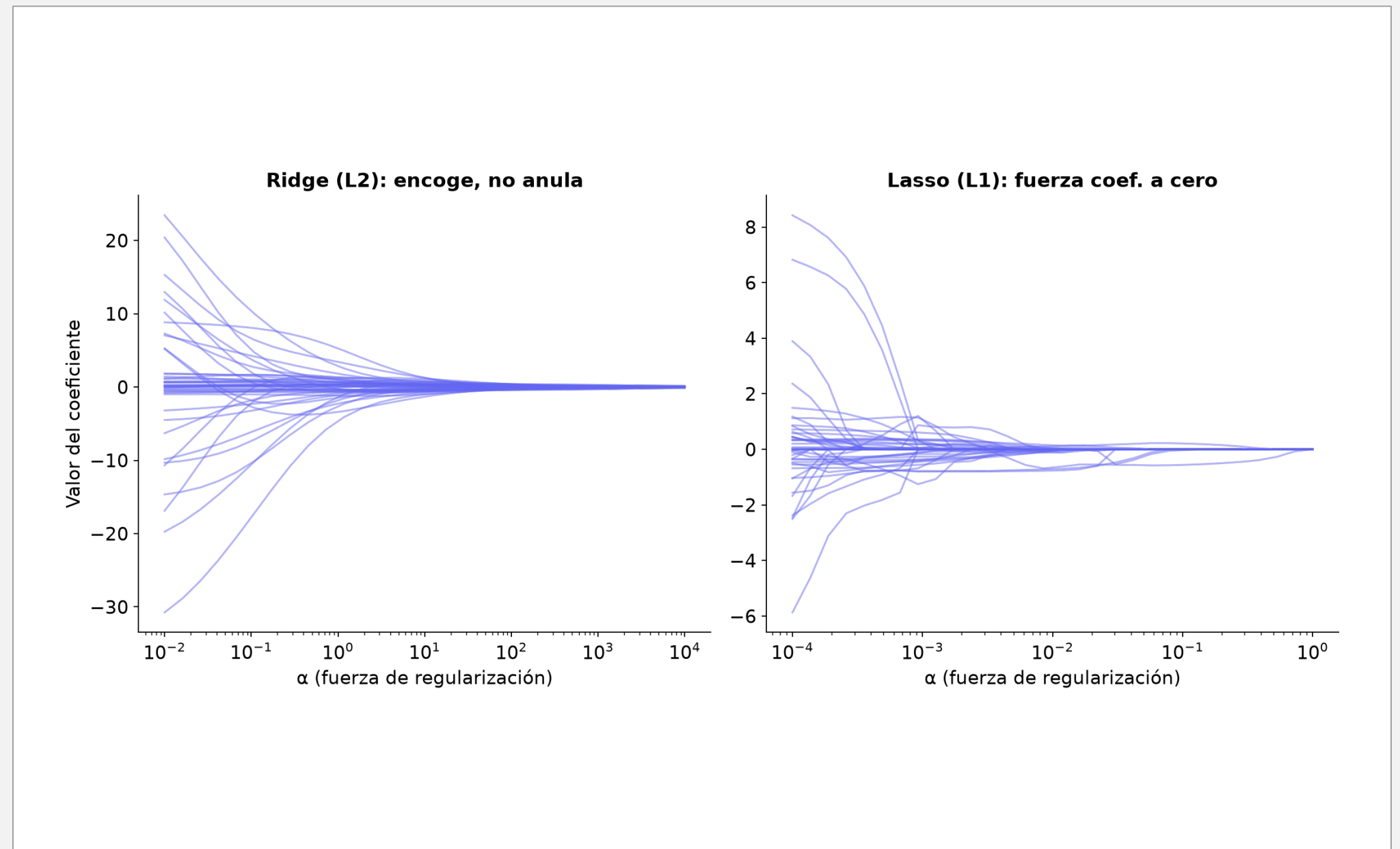
**Aprendizaje Supervisado:
Regresión Avanzada**

Sobreajuste y regularización

- Un ejemplo sintético 1D muestra el sobreajuste de forma aislada: un polinomio de grado alto se ajusta perfecto a los puntos de entrenamiento, pero oscila entre ellos
- Sobre California Housing (expandido con PolynomialFeatures de grado 2) se comparan tres formas de regularizar:
- Ridge (L2): penaliza el cuadrado de los coeficientes, los encoge sin anularlos
- Lasso (L1): penaliza el valor absoluto — puede llevar coeficientes a exactamente cero
- ElasticNet: combina L1 y L2 en una sola penalización

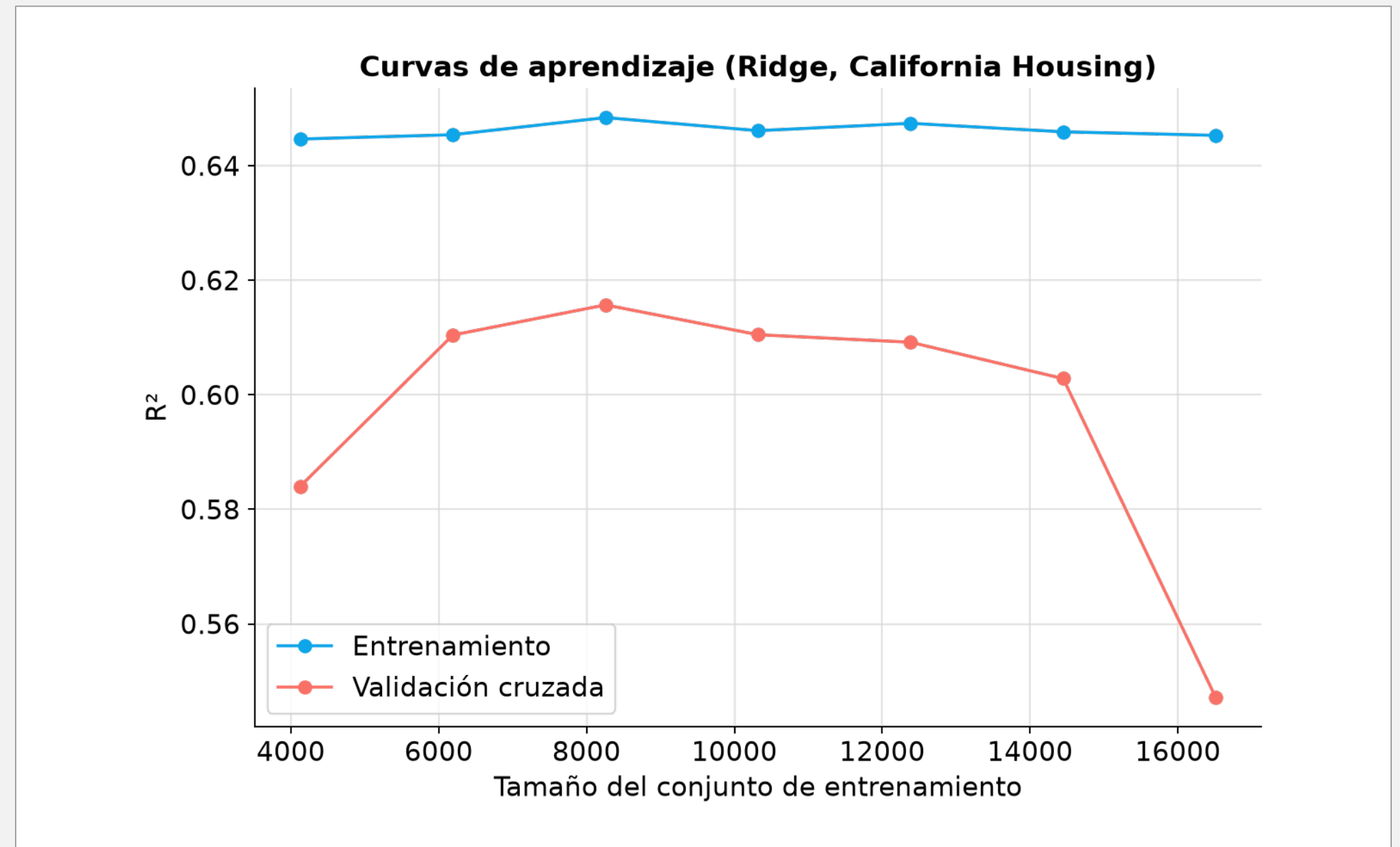
El efecto real de Ridge vs. Lasso

- Mismos datos (California Housing + interacciones polinomiales) — cada panel usa el rango de α donde su propio efecto se aprecia mejor
- Lasso lleva varios coeficientes a exactamente cero a medida que α crece; Ridge los encoge, pero rara vez llega a cero



Curvas de aprendizaje reales (Ridge)

- Brecha entre entrenamiento y validación cruzada: diagnóstico visual de sesgo/varianza
- Calculado sobre datos reales de California Housing



08

Aprendizaje Supervisado: Clasificación Avanzada

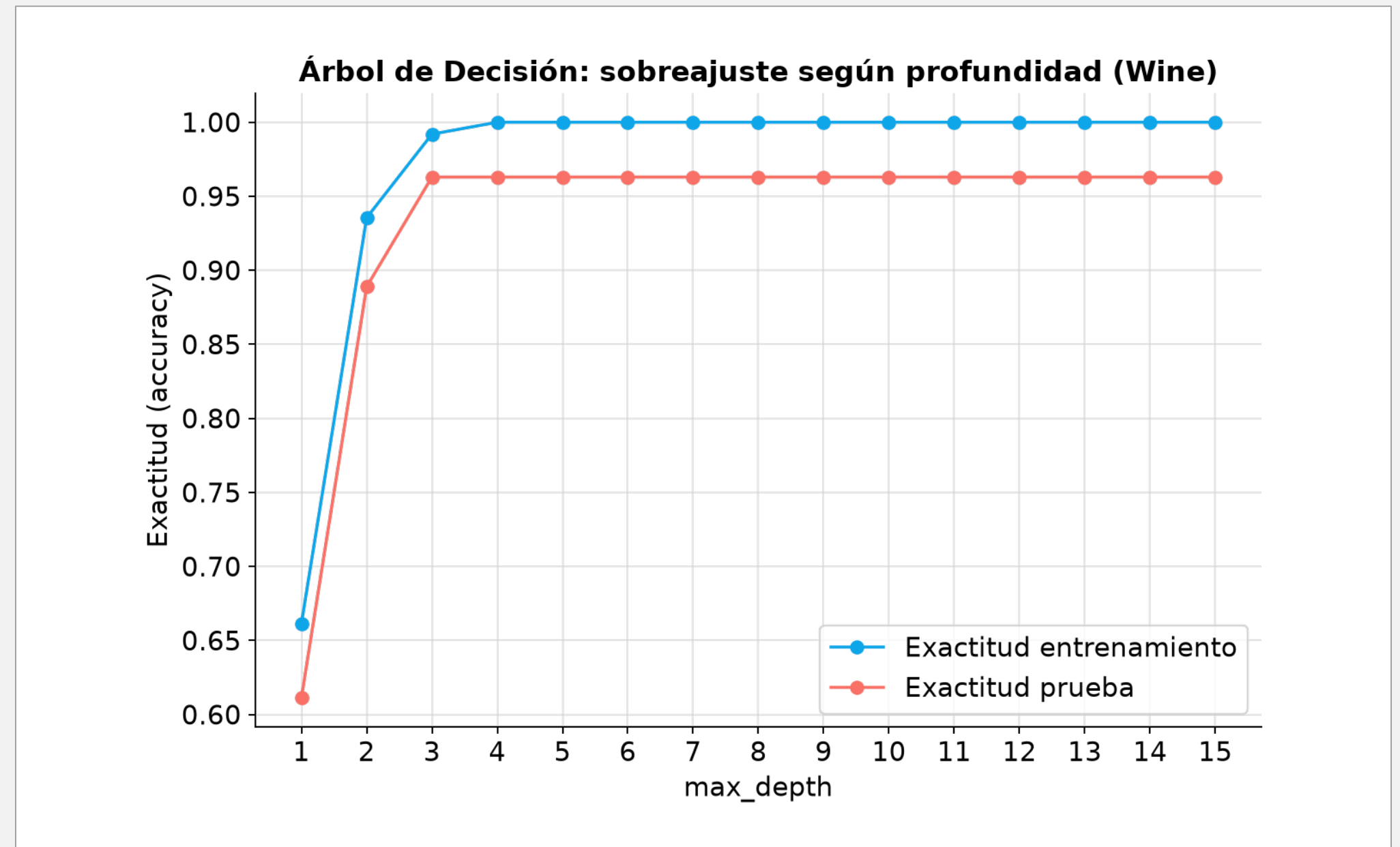
Por qué Wine y no Iris, esta vez

Iris tiene solo 4 variables y sus 3 clases están casi perfectamente separadas — varios modelos llegan al 100% de exactitud sin esfuerzo. Este notebook usa en su lugar Wine (13 variables), donde sí se aprecian diferencias entre modelos.

Modelo	Idea central
Regresión logística (multiclase)	Extiende la regresión logística binaria a más de dos clases
SVM (kernel RBF)	Busca el hiperplano (o frontera no lineal) que separa las clases con el mayor margen posible
Árbol de Decisión	Se muestra su curva de sobreajuste: a mayor profundidad, más se ajusta al ruido

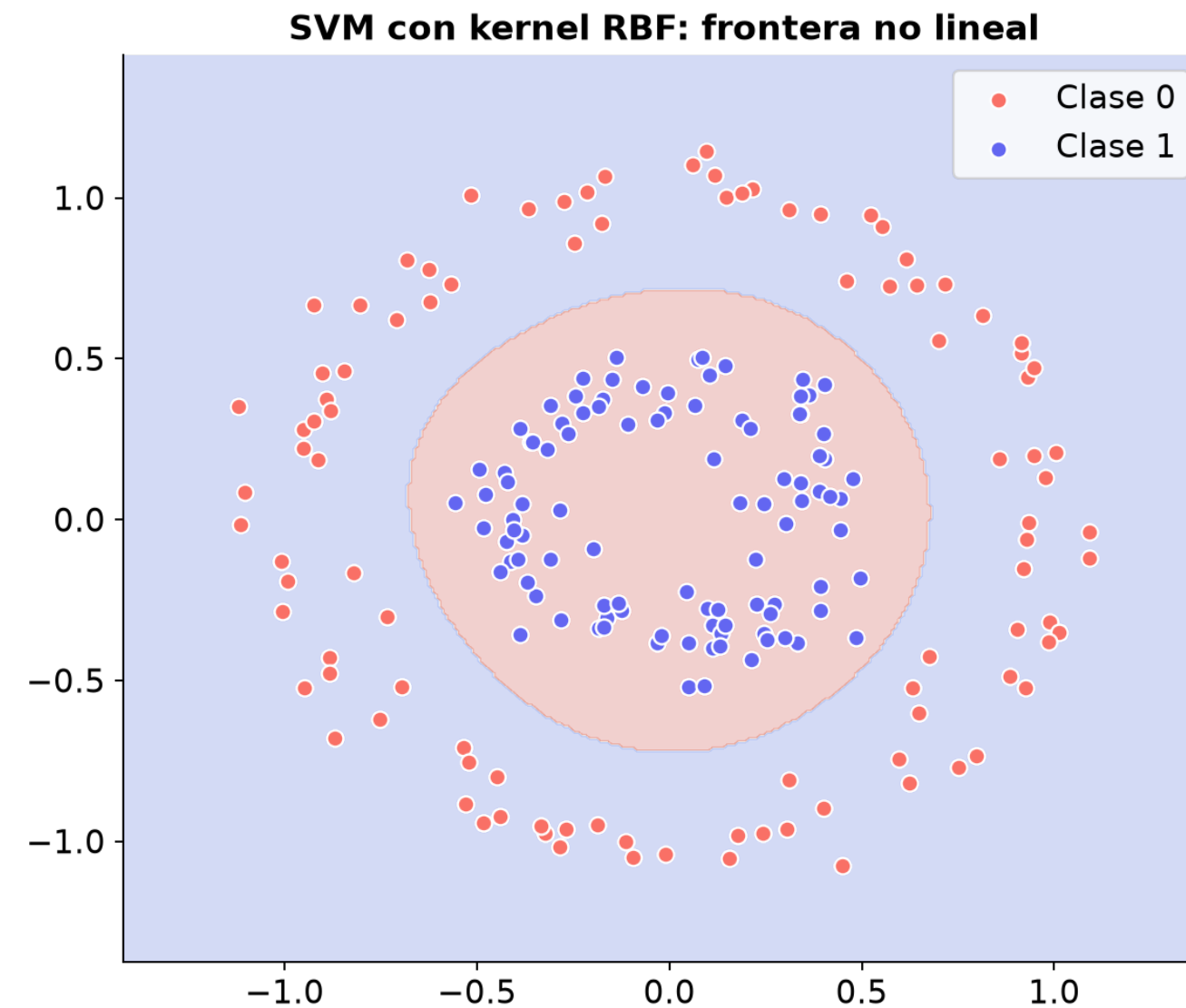
Árbol de Decisión: sobreajuste real, sobre Wine

- $\text{max_depth} \approx 3-4$: mejor exactitud de prueba
- Profundidades mayores solo memorizan el entrenamiento



SVM: fronteras no lineales con el kernel RBF

- Ejemplo ilustrativo en 2D: un kernel lineal no podría separar estas dos clases
- El kernel RBF proyecta implícitamente a más dimensiones, donde sí existe un separador simple



Comparación final con validación cruzada

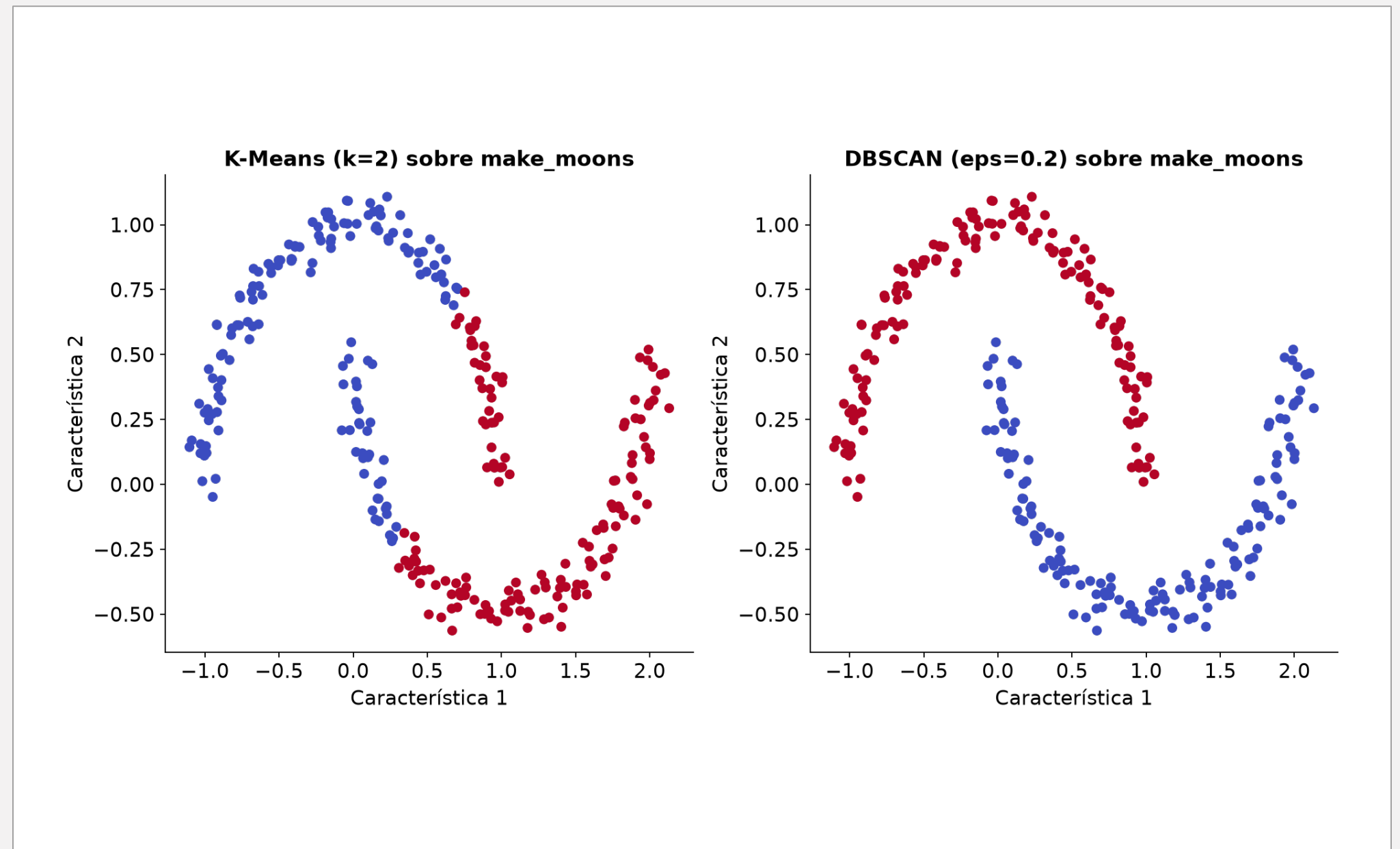
- En el notebook principal, k-NN y los Árboles se evaluaron con una única partición de entrenamiento y prueba
- Aquí se comparan cuatro modelos con `cross_val_score` (5 particiones), sobre las 13 variables de Wine: Regresión Logística, SVM (kernel RBF), Árbol de Decisión y k-NN (k=5)
- Todos con la misma preparación de datos (StandardScaler) — una comparación justa entre algoritmos

09

**Aprendizaje
No Supervisado**

K-Means falla, DBSCAN lo resuelve

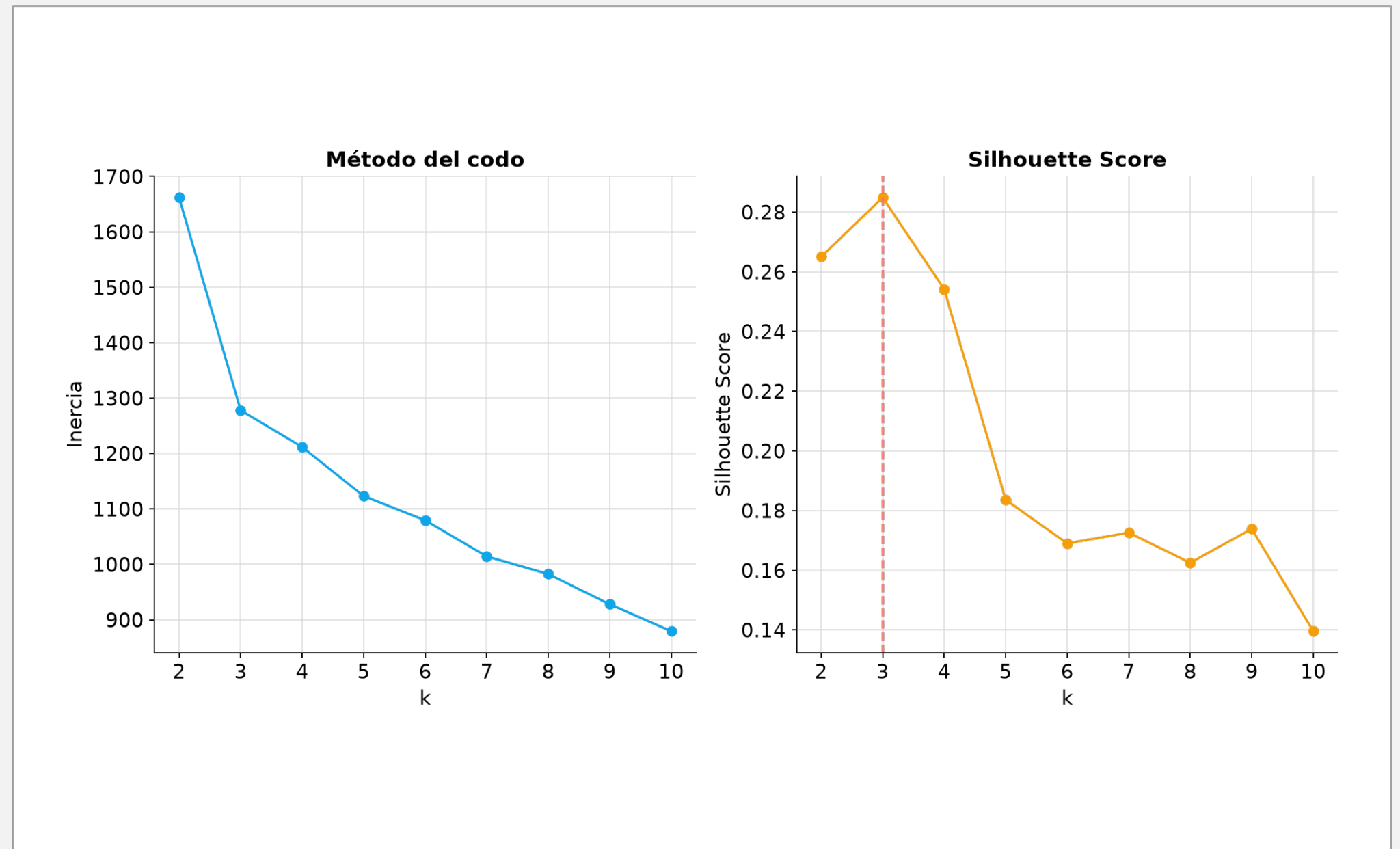
- Mismos datos (make_moons), mismo $k=2$ — K-Means traza una frontera recta que corta cada media luna por la mitad
- DBSCAN agrupa por densidad, no por cercanía a un centroide, y sí recupera la forma real de los dos grupos



Elegir k de forma objetiva, sobre Wine real

El mayor Silhouette Score real ocurre en $k=3$ (score=0.285) — coincide exactamente con las 3 variedades de uva reales del dataset Wine

El codo (inercia) es más ambiguo aquí; el Silhouette Score da una respuesta más objetiva



10

**Modelos de
Ensamble**

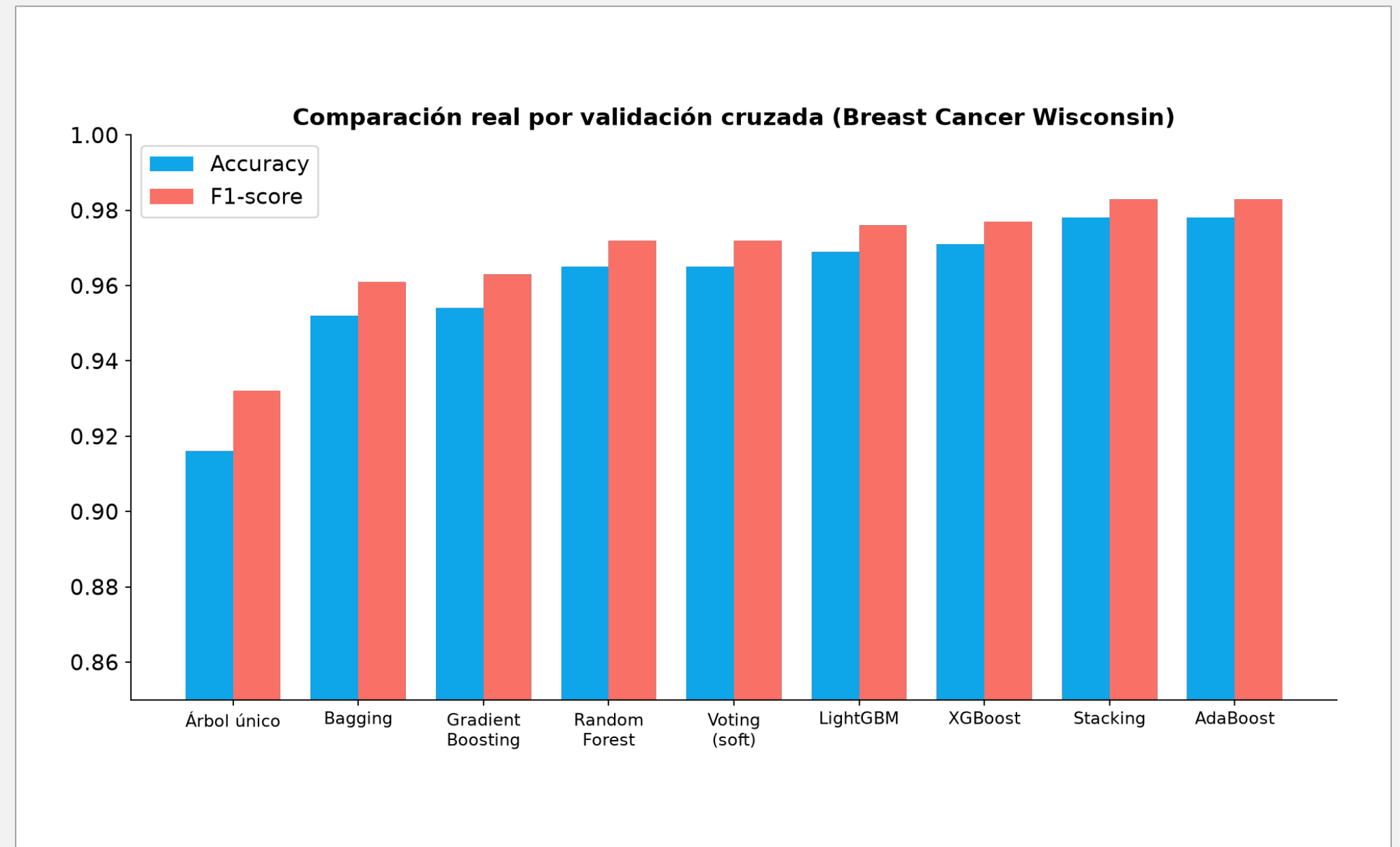
Bagging, Boosting, Voting y Stacking

Familia	Modelos vistos
Bagging	Bagging genérico, Random Forest (con importancia de características)
Boosting	AdaBoost, Gradient Boosting, XGBoost, LightGBM
Voting	Combina las predicciones de varios modelos ya entrenados (votación dura o suave)
Stacking	Un meta-modelo aprende a combinar las salidas de los modelos base

Todo se evalúa sobre el mismo dataset — Breast Cancer Wisconsin (diagnóstico de tumores).

Comparación real, por validación cruzada

- Hasta el ensamble más simple (Bagging) supera claramente al árbol único
- AdaBoost y Stacking lideran con ROC-AUC ≈ 0.995



11

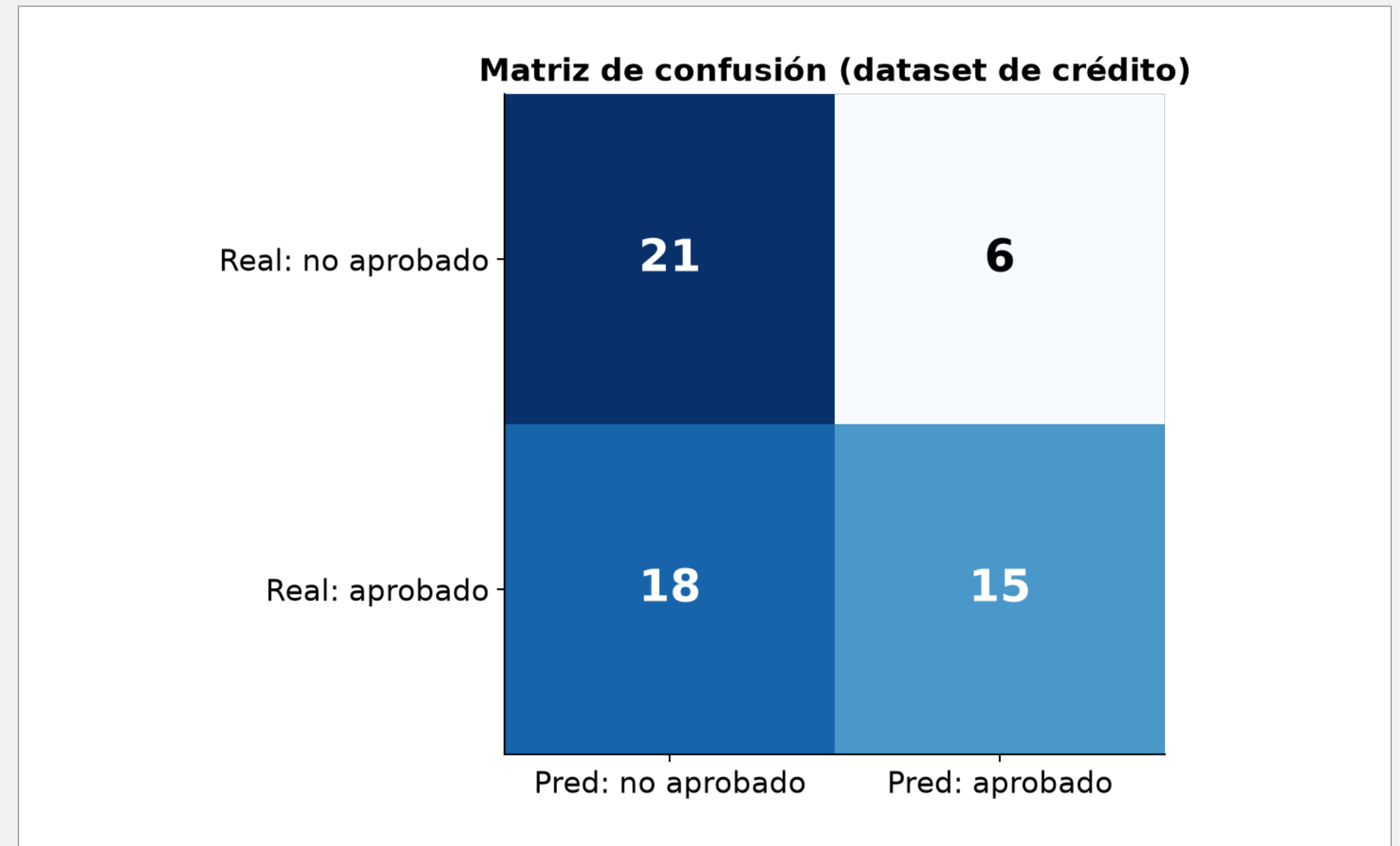
Pipelines y Evaluación de Modelos

Pipeline y ColumnTransformer

- Un Pipeline encadena pasos (imputar → escalar → codificar → clasificar) como un único estimador con fit/predict/score
- Evita la fuga de datos por construcción: cada transformación se ajusta solo con los datos de entrenamiento
- ColumnTransformer aplica transformaciones distintas según el tipo de columna, en el dataset de crédito bancario:
numéricas (edad, ingreso, puntaje) → imputar + escalar · categóricas (ciudad, nivel educativo) → codificar
- Sobre ese Pipeline completo se aplican validación cruzada y GridSearchCV

Métricas reales, más allá de accuracy

- Clase "aprobado": precisión 0.71, recall 0.45 ·
Clase "no aprobado": precisión 0.54, recall 0.78
- Accuracy global: 0.60 — ROC-AUC: 0.692
- La accuracy global no deja ver que el modelo detecta solo el 45% de las solicitudes que debían aprobarse



Clases desbalanceadas y SMOTE

- Con un dataset deliberadamente desbalanceado (95% / 5%), un clasificador "ingenuo" alcanza ~95% de accuracy sin haber aprendido nada
- SMOTE genera ejemplos sintéticos de la clase minoritaria, interpolando entre vecinos reales, en vez de solo duplicarlos
- Curvas de aprendizaje y de validación: mismo diagnóstico de sesgo/varianza de la Unidad 3.7, ahora dentro de un Pipeline completo

Resumen de la Unidad 3

Tema	Notebook del curso
Álgebra lineal, gradiente, Bayes	1-AI-fundamentals/1, 2, 3
EDA y preprocesamiento	1-AI-fundamentals/4, 5
Recorrido completo de ML	2-introduccion-machine-learning.ipynb
Regresión y clasificación avanzada	3-aprendizaje-supervisado/1, 2
Aprendizaje no supervisado	4-aprendizaje-no-supervisado.ipynb
Modelos de ensamble	5-modelos-ensamble.ipynb
Pipelines y evaluación	6-pipelines-evaluacion.ipynb

¡Gracias!

Estos algoritmos y buenas prácticas son la base del Machine Learning clásico — y la comparación directa frente a la Unidad 4 (redes neuronales).